Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Уфимский государственный нефтяной технический университет»

Кафедра «Цифровые технологии и моделирование»

09.04.01 Информатика и вычислительная техника

РАЗРАБОТКА VR-ТРЕНАЖЁРА С БИОЛОГИЧЕСКОЙ ОБРАТНОЙ СВЯЗЬЮ ДЛЯ РЕАБИЛИТАЦИИ ПОСЛЕ ИНСУЛЬТА

Курсовая работа

Работа защищена:

« »_____2026 г.

Оценка_____
(подпись)

Выполнил студент
2 курса МПИ09-25-01 группы

Музафаров И.Ф.

_____
(ф.и.о., подпись)

Научный руководитель

_____
(уч. Степень, уч.звание, фамилия, инициалы)

_____
(подпись)

Уфа 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ НЕЙРОВИРТУАЛЬНЫХ ИНТЕРФЕЙСОВ 5
1.1 Нейроинтерфейсы и моторное воображение 5
1.2 Обработка ЭЭГ и виртуальная реальность в реабилитации 6
2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОТОТИПА 8
2.1 Постановка задачи и архитектура системы 8
2.2 Подготовка данных и разработка классификатора 9
3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ VR-ТРЕНАЖЁРА 12
3.1 Серверная часть и интеграция с Unity 12
3.2 Функциональное тестирование и ограничения прототипа 14
ЗАКЛЮЧЕНИЕ 17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 18
ПРИЛОЖЕНИЕ А 21

ВВЕДЕНИЕ

Инсульт часто приводит к нарушениям двигательных функций, из-за которых человеку становится сложнее выполнять повседневные действия. Для восстановления движений применяются различные методы реабилитации, в том числе технологии виртуальной реальности и нейроинтерфейсы. Интерфейсы «мозг — компьютер» (Brain-Computer Interface, BCI) позволяют преобразовывать сигналы мозговой активности в команды для управления внешними устройствами [1, 2].

Одним из направлений применения таких интерфейсов является восстановление двигательных функций верхних конечностей после инсульта. В частности, исследуется использование моторного воображения для управления реабилитационными упражнениями [3, 4]. Для регистрации мозговой активности может применяться электроэнцефалография (ЭЭГ), а виртуальная реальность позволяет отображать движения и результаты выполнения заданий в интерактивной среде [5–7].

Цель курсовой работы — разработать программный прототип VR-тренажёра для реабилитационных упражнений верхних конечностей после инсульта с использованием классификации записей ЭЭГ для управления виртуальной рукой.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Рассмотреть принципы работы нейроинтерфейсов и методы распознавания моторного воображения.
2. Изучить методы обработки и классификации сигналов ЭЭГ, используемые в проекте.
3. Разработать архитектуру программного прототипа и подготовить данные для классификации.
4. Реализовать передачу результатов классификации из Python в Unity через серверное приложение.

5. Реализовать управление виртуальной рукой и отображение результатов выполнения упражнения.
6. Провести функциональное тестирование прототипа и определить ограничения его текущей реализации.

Объект исследования — нейровиртуальные интерфейсы, использующие сигналы мозговой активности для управления виртуальными объектами.

Предмет исследования — методы обработки и классификации сигналов ЭЭГ, а также способы интеграции результатов классификации с виртуальной средой.

В работе используются методы анализа научной литературы, цифровой обработки сигналов, машинного обучения, программной интеграции и функционального тестирования.

Практическая значимость заключается в создании программного прототипа, объединяющего обработку записей ЭЭГ, передачу классифицированных команд и управление виртуальной рукой. В текущей реализации используются заранее записанные данные, поэтому прототип не обеспечивает управление по непрерывному сигналу ЭЭГ в реальном времени. Также обратная связь на основе физиологических показателей пока не реализована полностью.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе рассматриваются теоретические основы нейровиртуальных интерфейсов. Во второй описываются проектирование системы, подготовка данных и разработка классификатора. В третьей рассматриваются интеграция компонентов, функциональное тестирование и ограничения прототипа.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ НЕЙРОВИРТУАЛЬНЫХ ИНТЕРФЕЙСОВ

1.1 Нейроинтерфейсы и моторное воображение

Нейроинтерфейс «мозг — компьютер» (Brain–Computer Interface, BCI) представляет собой систему, которая преобразует сигналы мозговой активности в команды для внешних устройств. В реабилитации после инсульта такие системы исследуются как один из способов восстановления двигательных функций, в том числе с использованием мысленного представления движений [3, 4].

Одним из подходов является регистрация электроэнцефалограммы (ЭЭГ). Она позволяет измерять электрическую активность мозга с помощью электродов, расположенных на поверхности головы. Полученные сигналы содержат полезную информацию, но также могут включать шумы и артефакты. Поэтому перед распознаванием команд данные необходимо предварительно обработать [1, 2, 6].

Моторное воображение — это мысленное представление движения без его фактического выполнения. При воображении движений левой и правой рукой изменяется активность сенсомоторных областей мозга. В частности, при выполнении таких задач могут наблюдаться изменения синхронизации и десинхронизации ритмов ЭЭГ. Эти особенности используются для различения классов моторного воображения [8, 9, 19].

Для распознавания команд применяются алгоритмы машинного обучения, которые выявляют различия между характеристиками сигналов. Выбор метода обработки и классификации влияет на работу BCI, поэтому при разработке подобных систем необходимо учитывать особенности данных и условия проведения эксперимента [7, 18].

В данной курсовой работе рассматриваются два класса моторного воображения: движение левой и правой рукой. Для их распознавания

используется классификатор, обученный на заранее записанных данных ЭЭГ. Это позволяет проверить передачу команд и управление виртуальной рукой без подключения физического электроэнцефалографа.

1.2 Обработка ЭЭГ и виртуальная реальность в реабилитации

Перед использованием сигналов ЭЭГ для управления виртуальными объектами необходимо выполнить их предварительную обработку. Она включает выбор нужных каналов, фильтрацию сигнала и выделение временных участков, соответствующих выполнению конкретной задачи. После этого из данных извлекаются признаки, которые используются для классификации моторного воображения [6, 8, 15–17].

В данной работе для распознавания воображаемых движений применяется метод общих пространственных шаблонов (Common Spatial Patterns, CSP) совместно с линейным дискриминантным анализом (Linear Discriminant Analysis, LDA). CSP позволяет получить пространственные признаки, различающиеся между двумя классами сигналов ЭЭГ, а LDA используется для отнесения полученных данных к одному из классов [8, 18, 23].

В качестве источника данных используется открытый набор ЭЭГ, содержащий записи моторного воображения у пациентов в остром периоде инсульта [15, 20]. В программном прототипе анализируются два класса: воображение движения левой и правой рукой. Полученный результат классификации передаётся в серверное приложение, а затем используется для управления виртуальной рукой в Unity.

Виртуальная реальность позволяет моделировать упражнения в контролируемой среде и визуально отображать действия пользователя. Исследования показывают, что VR-технологии применяются в реабилитации после инсульта, в том числе для тренировки движений верхних конечностей [10–14]. В разработанном прототипе виртуальная рука перемещается к

целевому объекту, а интерфейс отображает попадание в цель и количество выполненных повторений.

При этом необходимо различать обратную связь о выполнении упражнения и биологическую обратную связь, основанную на физиологических показателях. В текущей реализации отображаются результаты виртуального упражнения, однако отдельный механизм обратной связи на основе параметров ЭЭГ ещё не реализован. Кроме того, используются заранее записанные данные, поэтому управление не является непрерывным и не выполняется в реальном времени.

Таким образом, сочетание обработки ЭЭГ, классификации моторного воображения и виртуальной среды позволяет создать программный прототип для исследования управления реабилитационным упражнением. Описанные методы составляют теоретическую основу дальнейшего проектирования и реализации системы.

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОТОТИПА

2.1 Постановка задачи и архитектура системы

Цель разработки — создать программный прототип VR-тренажёра, в котором результаты классификации записей ЭЭГ используются для управления виртуальной рукой при выполнении реабилитационного упражнения. Система должна обеспечивать обработку данных, передачу распознанных команд и отображение результата в Unity.

Архитектура прототипа включает три основных компонента: модуль обработки ЭЭГ и классификации на Python, серверное приложение на FastAPI и виртуальную среду, разработанную в Unity. Компоненты взаимодействуют через WebSocket-соединения, позволяющие передавать сообщения между программами без необходимости создавать отдельный HTTP-запрос для каждого сообщения [21, 22].

Модуль Python обрабатывает заранее записанные сигналы ЭЭГ и определяет предполагаемый класс моторного воображения. Результат передаётся серверу FastAPI, который принимает сообщение, сохраняет последние полученные данные и пересылает их подключённым клиентам Unity. Сервер также предоставляет HTTP-эндпоинты для проверки состояния приложения и получения последних данных.

Клиент Unity получает команду и передаёт её контроллеру виртуальной руки. Контроллер управляет движением руки к целевому объекту и фиксирует результат упражнения. Пользовательский интерфейс отображает текущую команду, состояние упражнения и количество успешно выполненных повторений (см. рисунок 2.1).

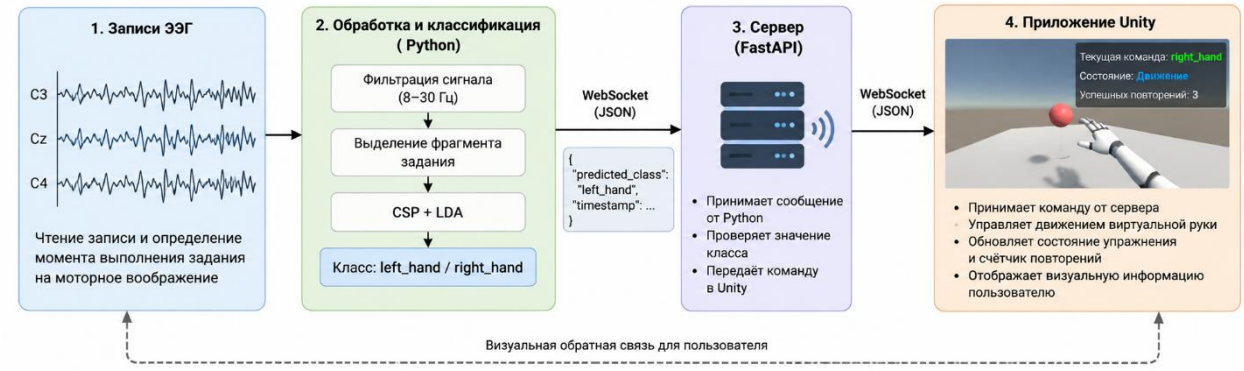


Рисунок 2.1 — Архитектура программного прототипа

Таким образом, архитектура разделяет обработку сигналов, передачу сообщений и визуализацию. Это упрощает тестирование отдельных компонентов и позволяет в дальнейшем заменить воспроизведение записей ЭЭГ потоком данных от реального устройства.

2.2 Подготовка данных и разработка классификатора

Для обучения классификатора используется открытый набор данных NEMAR, содержащий записи ЭЭГ пациентов после инсульта при выполнении задач моторного воображения [15, 20]. В проекте данные загружаются из файлов формата BDF, а информация о событиях используется для определения временных участков, соответствующих экспериментальным заданиям.

Перед классификацией выполняется предварительная обработка сигналов. В программе выбираются три канала ЭЭГ: C3, Cz и C4, расположенные над сенсомоторными областями мозга. Затем применяется полосовой фильтр с диапазоном частот 8–30 Гц. Для каждого события выделяется эпоха длительностью три секунды — от 0,5 до 3,5 секунды после начала события. В результате формируются фрагменты сигналов, соответствующие двум классам: воображению движения левой и правой рукой.

Для извлечения признаков используется метод общих пространственных шаблонов (CSP). Он преобразует исходные сигналы каналов в пространственные признаки, позволяющие различать два класса моторного воображения. Затем линейный дискриминантный анализ (LDA) определяет класс нового фрагмента ЭЭГ. Совместное применение CSP и LDA позволяет построить классификатор для решения задачи бинарного распознавания [8, 18, 23].

Обучение модели реализовано в скрипте train_eeg.py. Для оценки обобщающей способности используется схема Leave-One-Group-Out: на каждом шаге один участник исключается из обучающей выборки и используется для проверки модели. Такое разделение позволяет оценить работу классификатора на данных человека, не участвовавшего в обучении. После обучения модель сохраняется в файл motor_imagery_model.joblib и используется при последующем воспроизведении записей ЭЭГ.

Для проверки программной цепочки скрипт replay_eeg.py загружает запись участника, последовательно обрабатывает выделенные эпохи и получает предсказания классификатора. Каждое предсказание содержит определённый класс моторного воображения и передаётся серверу по WebSocket. При этом скрипт выводит как предсказанный класс, так и фактическую метку события, что позволяет сопоставлять результаты распознавания с исходными данными (см. рисунок 2.2).

Таким образом, подготовка данных и классификация обеспечивают получение команд для управления виртуальной рукой. В текущей реализации классификатор работает с заранее записанными данными, которые воспроизводятся последовательно, а не с непрерывным потоком ЭЭГ в реальном времени.

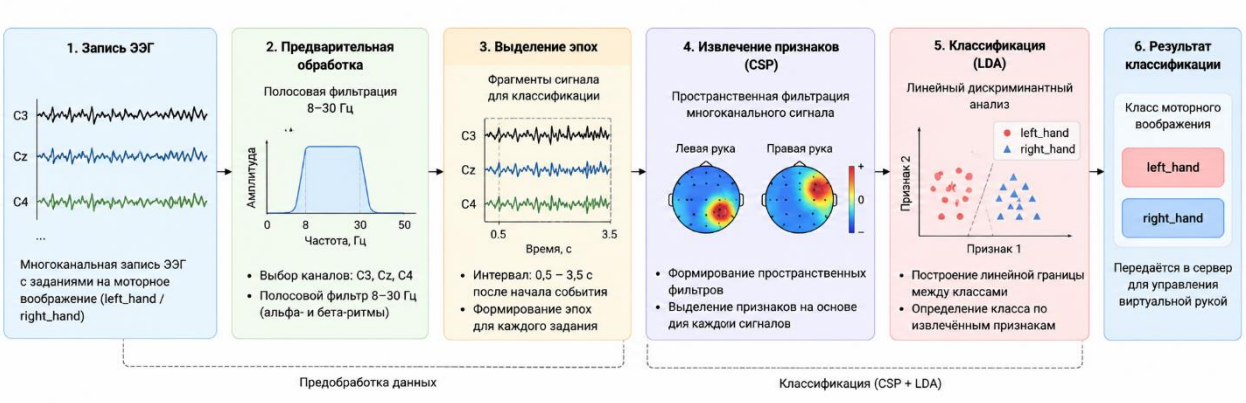


Рисунок 2.2 — Этапы обработки сигналов ЭЭГ и классификации моторного воображения

3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ VR-ТРЕНАЖЁРА

3.1 Серверная часть и интеграция с Unity

Серверная часть разработана на FastAPI и обеспечивает обмен данными между модулем обработки ЭЭГ и виртуальной средой. Для передачи команд используются WebSocket-соединения, реализованные в серверном приложении и клиенте Unity [21, 22].

Сервер предоставляет обработчики /ws/python и /ws/unity. Первый принимает сообщения от скрипта воспроизведения записей ЭЭГ, второй используется для подключения Unity. Полученная команда содержит класс моторного воображения — left_hand или right_hand, а также дополнительные сведения об источнике и времени передачи. Сервер сохраняет последние полученные данные и пересылает сообщения подключённым клиентам. Для проверки доступности приложения предусмотрен эндпоинт /health, а для получения последних данных — /api/latest.

В Unity получение сообщений реализовано в скрипте FastAPIAlphaReceiver.cs. Он подключается к серверу, считывает JSON-сообщения и передаёт распознанный класс контроллеру упражнения через метод SetEEGCommand(). Взаимодействие организовано таким образом, чтобы получение сетевых сообщений не блокировало обработку игровой сцены.

Основная логика упражнения находится в скрипте RehabArmController.cs. Он управляет виртуальной рукой, перемещением к целевому объекту и подсчётом успешных повторений. При достижении цели тренажёр фиксирует результат, изменяет состояние мишени и возвращает руку в исходное положение. Для проверки работы сцены также предусмотрено ручное управление клавишами Q и E, а клавиша R возвращает руку в исходное положение.

Для отображения состояния упражнения используется RehabFeedbackUI.cs. Интерфейс показывает текущую команду, статус выполнения, прогресс и количество завершённых повторений. После достижения заданного количества успешных повторений тренажёр сообщает о завершении упражнения.

Внешний вид виртуальной среды и панели обратной связи представлен на рисунке 3.1.

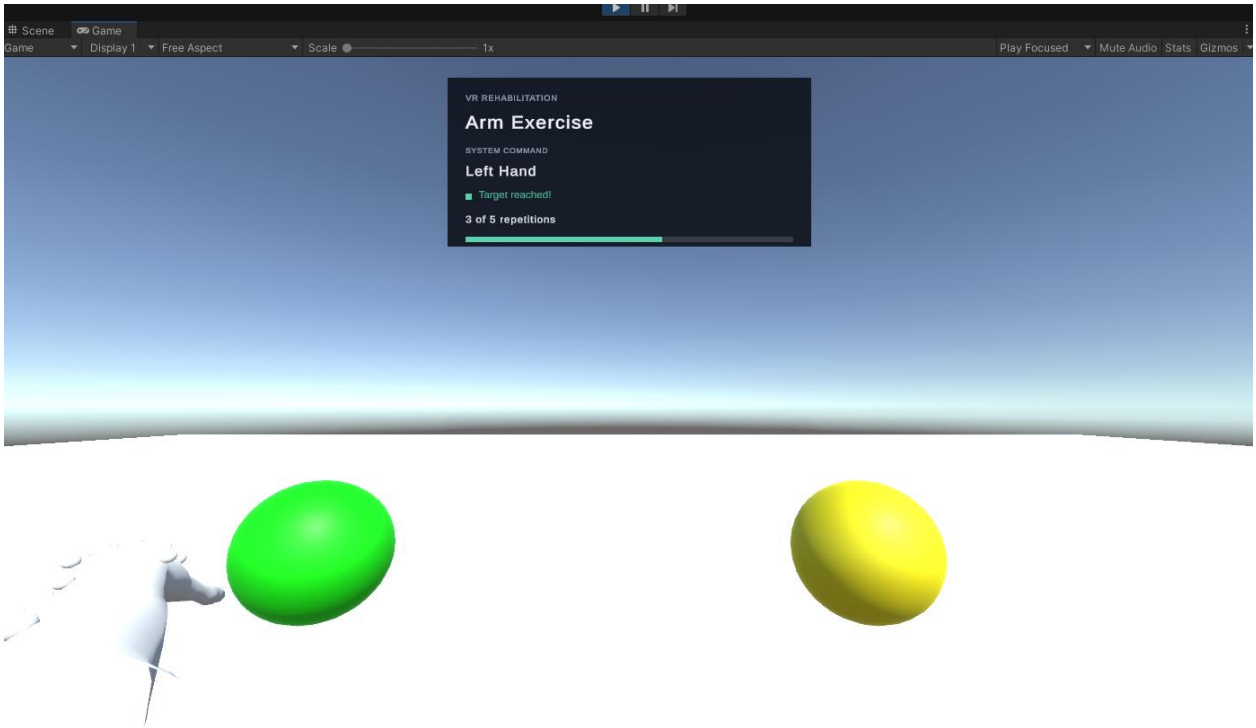


Рисунок 3.1 — Виртуальная среда реабилитационного упражнения

Программный код серверного приложения, обеспечивающего обмен данными между модулем обработки ЭЭГ и виртуальной средой, представлен в приложении А.

Таким образом, реализован обмен классифицированными командами между Python, сервером FastAPI и Unity. При этом управление проверяется на заранее записанных данных, а не на непрерывном сигнале от физического ЭЭГ-устройства.

3.2 Функциональное тестирование и ограничения прототипа

Функциональное тестирование проводилось для проверки взаимодействия основных компонентов программного прототипа: классификатора на Python, серверного приложения FastAPI и VR-сцены в Unity. Основное внимание уделялось передаче команд, реакции виртуальной руки и отображению результатов упражнения.

В ходе проверки рассматривались следующие сценарии:

1. Запуск серверного приложения и проверка доступности его эндпоинтов.
2. Подключение Python-клиента и Unity к соответствующим WebSocket-адресам.
3. Передача предсказаний классов left_hand и right_hand.
4. Проверка реакции виртуальной руки на полученную команду.
5. Проверка регистрации попадания в целевой объект и увеличения счётчика успешных повторений.
6. Проверка отображения текущего состояния упражнения и завершения тренировки после достижения заданного количества повторений.

Результат выполнения упражнения отображается в пользовательском интерфейсе. При достижении цели тренажёр фиксирует успешное повторение, обновляет прогресс и возвращает виртуальную руку в исходное положение. Если цель не достигнута, интерфейс отображает соответствующее состояние. Дополнительно предусмотрено ручное управление, позволяющее проверять работу виртуальной сцены независимо от классификатора.

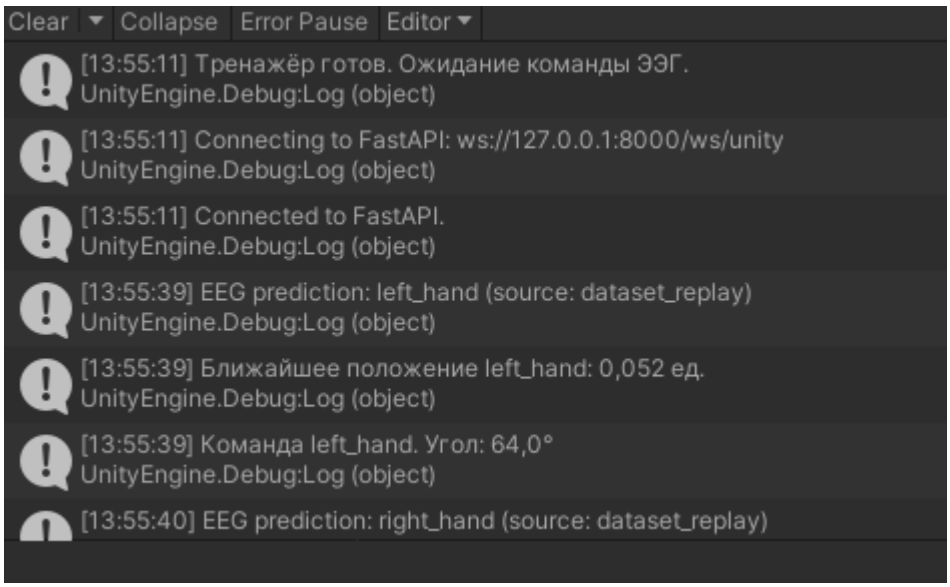


Рисунок 3.2 — Проверка взаимодействия приложения Unity с сервером FastAPI.

Текущая реализация имеет ряд ограничений. Во-первых, команды формируются на основе заранее записанных данных ЭЭГ, поэтому система не обеспечивает непрерывного управления в реальном времени. Во-вторых, результаты классификации используются для управления виртуальной рукой, но отдельная физиологическая обратная связь на основе параметров ЭЭГ не реализована: метод SetEEGFeedback() в текущей версии не содержит соответствующей логики. В-третьих, функциональное тестирование программной цепочки само по себе не подтверждает клиническую эффективность тренажёра.

Для дальнейшего развития проекта можно предусмотреть подключение физического ЭЭГ-устройства, обработку непрерывного потока данных, реализацию обратной связи на основе выбранных физиологических показателей и расширенную оценку качества классификации.

Таким образом, реализован программный прототип, объединяющий обработку записей ЭЭГ, передачу результатов классификации и управление виртуальным реабилитационным упражнением. Проведённые функциональные проверки позволяют оценить работоспособность основных

компонентов, а выявленные ограничения определяют направления дальнейшего развития системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан программный прототип VR-тренажёра для реабилитационных упражнений верхних конечностей после инсульта с использованием классификации записей ЭЭГ для управления виртуальной рукой.

В процессе работы были рассмотрены принципы построения нейроинтерфейсов и применения моторного воображения, изучены методы предварительной обработки и классификации сигналов ЭЭГ. Для распознавания воображаемых движений левой и правой рукой реализована обработка записей ЭЭГ с использованием метода CSP и классификатора LDA.

Разработана архитектура программного прототипа, объединяющая модуль обработки данных на Python, серверное приложение FastAPI и виртуальную среду Unity. Передача результатов классификации осуществляется по WebSocket. В Unity реализованы управление виртуальной рукой, взаимодействие с целевым объектом, подсчёт успешных повторений и отображение состояния упражнения.

Функциональная проверка предусмотренных сценариев позволяет оценить взаимодействие программных компонентов и работу основных элементов тренажёра. При этом текущая версия использует заранее записанные данные ЭЭГ и не обеспечивает непрерывного управления в реальном времени. Также в ней отсутствует полноценная биологическая обратная связь, основанная на физиологических показателях.

Таким образом, поставленная цель достигнута: создан программный прототип, демонстрирующий возможность объединения классификации записей ЭЭГ с управлением реабилитационным упражнением в виртуальной среде. Дальнейшее развитие проекта может включать подключение физического ЭЭГ-устройства, обработку сигналов в реальном времени и реализацию физиологической обратной связи. Для оценки практической эффективности тренажёра также потребуются дополнительные испытания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lotte F., Congedo M., Lécuyer A., Lamarche F., Arnaldi B. A review of classification algorithms for EEG-based brain-computer interfaces // *Journal of Neural Engineering*. 2007. Vol. 4, no. 2. P. R1–R13. DOI: https://doi.org/10.1088/1741-2560/4/2/R01.
2. Abiri R., Borhani S., Sellers E. W., Jiang Y., Zhao X. A comprehensive review of EEG-based brain-computer interface paradigms // *Journal of Neural Engineering*. 2019. Vol. 16, no. 1. Article 011001. DOI: https://doi.org/10.1088/1741-2552/aaf12e.
3. Панков М. Ю., Костенко Е. В., Петрова Л. В., Филиппов М. С. Интерфейс мозг–компьютер в восстановлении двигательной функции верхней конечности после инсульта // *Неврология, нейропсихиатрия, психосоматика*. 2025. Т. 17, № 2. С. 93–99. DOI: https://doi.org/10.14412/2074-2711-2025-2-93-99.
4. Khan M. A., Das R., Iversen H. K., Puthusserypady S. Review on motor imagery based BCI systems for upper limb post-stroke neurorehabilitation: From designing to application // *Computers in Biology and Medicine*. 2020. Vol. 123. Article 103843. DOI: https://doi.org/10.1016/j.compbiomed.2020.103843.
5. Люкманов Р. Х., Исаев М. Р., Мокиенко О. А., Бобров П. Д., Иконникова Е. С., Черкасова А. Н., Супонева Н. А. Интерфейс мозг–компьютер, основанный на спектроскопии в ближней инфракрасной области, в двигательной реабилитации после инсульта: описание серии случаев // *Анналы клинической и экспериментальной неврологии*. 2023. Т. 17, № 4. С. 82–88. DOI: https://annaly-nevrologii.com/pathID/article/view/953.
6. Aggarwal S., Chugh N. Signal processing techniques for motor imagery brain computer interface: A review // *Array*. 2019. Vol. 1–2. Article 100003. DOI: https://doi.org/10.1016/j.array.2019.100003.

7. Lotte F. et al. A review of classification algorithms for EEG-based brain-computer interfaces: a 10 year update // *Journal of Neural Engineering*. 2018. Vol. 15, no. 3. Article 031005. DOI: https://doi.org/10.1088/1741-2552/aab2f2.
8. Müller-Gerking J., Pfurtscheller G., Flyvbjerg H. Designing optimal spatial filters for single-trial EEG classification in a movement task // *Clinical Neurophysiology*. 1999. Vol. 110, no. 5. P. 787–798. DOI: https://doi.org/10.1016/S1388-2457(98)00038-8.
9. Pfurtscheller G., Lopes da Silva F. H. Event-related EEG/MEG synchronization and desynchronization: basic principles // *Clinical Neurophysiology*. 1999. Vol. 110, no. 11. P. 1842–1857. DOI: https://doi.org/10.1016/S1388-2457(99)00141-8.
10. Chen J., Or C. K., Chen T. Effectiveness of Using Virtual Reality-Supported Exercise Therapy for Upper Extremity Motor Rehabilitation in Patients With Stroke: Systematic Review and Meta-analysis of Randomized Controlled Trials // *Journal of Medical Internet Research*. 2022. Vol. 24, no. 6. Article e24111. DOI: https://www.jmir.org/2022/6/e24111.
11. Ellis F. et al. Neurophysiological changes accompanying reduction in upper limb motor impairments in response to exercise-based virtual rehabilitation after stroke: systematic review // *Physiotherapy*. 2021. Vol. 113. P. 141–152. DOI: https://www.physiotherapyjournal.com/article/S0031-9406(21)00059-6/abstract.
12. Щербак С. Г., Вологжанин Д. А., Макаренко С. В., Голота А. С., Камилова Т. А. Виртуальная реальность в медицинской реабилитации // *Физическая и реабилитационная медицина, медицинская реабилитация*. 2025. Т. 7, № 2. С. 134–153. DOI: https://doi.org/10.36425/rehab677338.
13. Краснова-Гольева В. В., Гольев М. А. Виртуальная реальность в реабилитации после инсульта // *Современная зарубежная психология*. 2015. Т. 4, № 4. С. 39–44. DOI: https://doi.org/10.17759/jmfp.2015040406.
14. Laver K. E., Lange B., George S., Deutsch J. E., Saposnik G., Chapman M., Crotty M. Virtual reality for stroke rehabilitation // *Cochrane Database of Systematic Reviews*. 2025. Issue 6. Article CD008349. DOI:

https://www.cochranelibrary.com/cdsr/doi/10.1002/14651858.CD008349.pub5/full.

15. Liu H. et al. An EEG motor imagery dataset for brain computer interface in acute stroke patients // *Scientific Data*. 2024. Vol. 11. Article 131. DOI: https://doi.org/10.1038/s41597-023-02787-8.
16. Pernet C. R. et al. EEG-BIDS, an extension to the brain imaging data structure for electroencephalography // *Scientific Data*. 2019. Vol. 6. Article 103. DOI: https://www.nature.com/articles/s41597-019-0104-8.
17. Gramfort A. et al. MNE software for processing MEG and EEG data // *NeuroImage*. 2014. Vol. 86. P. 446–460. DOI: https://www.sciencedirect.com/science/article/abs/pii/S1053811913010501?via%3Dihub.
18. Pedregosa F. et al. Scikit-learn: Machine Learning in Python // *Journal of Machine Learning Research*. 2011. Vol. 12. P. 2825–2830. URL: https://jmlr.org/papers/v12/pedregosa11a.html.
19. Jeannerod M. Mental imagery in the motor context // *Neuropsychologia*. 1995. Vol. 33, no. 11. P. 1419–1432. DOI: https://www.sciencedirect.com/science/article/abs/pii/002839329500073C?via%3Dihub.
20. NEMAR. EEG dataset nm000158, version 1.0.5 [Data set]. URL: https://nemar.org/dataset/nm000158?v=v1.0.5.
21. FastAPI. WebSockets: official documentation. URL: https://fastapi.tiangolo.com/advanced/websockets/.
22. The WebSocket Protocol. RFC 6455. URL: https://www.rfc-editor.org/rfc/rfc6455.
23. Scikit-learn. LinearDiscriminantAnalysis: official documentation. URL: https://scikit-learn.org/stable/modules/generated/sklearn.discriminant_analysis.LinearDiscriminantAnalysis.html.

ПРИЛОЖЕНИЕ А

Main.py:

```
import asyncio
import math
from datetime import datetime, timezone

from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI(title="VR Rehabilitation API")

unity_clients: set[WebSocket] = set()
clients_lock = asyncio.Lock()

latest_data = {
    "alpha_power": None,
    "predicted_class": None,
    "timestamp": None,
    "source": "synthetic",
}

@app.get("/")
async def root():
    return {
        "message": "VR Rehabilitation API is running",
        "docs": "/docs",
    }

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.get("/api/latest")
async def get_latest():
    return latest_data.copy()

async def broadcast(data: dict):
    async with clients_lock:
        clients = list(unity_clients)

    disconnected = []

    for client in clients:
        try:
            await client.send_json(data)
        except Exception:
            disconnected.append(client)

    if disconnected:
        async with clients_lock:
            for client in disconnected:
                unity_clients.discard(client)

@app.websocket("/ws/python")
async def python_stream(websocket: WebSocket):
    global latest_data

    await websocket.accept()
    print("Python connected")

    try:
        while True:
            data = await websocket.receive_json()

            message = {
                "alpha_power": latest_data["alpha_power"],
                "predicted_class": latest_data["predicted_class"],
                "timestamp": datetime.now(timezone.utc).isoformat(),
                "source": data.get("source", "synthetic"),
            }

            # Alpha power is optional.
            if "alpha_power" in data:
                try:
                    alpha = float(data["alpha_power"])
                except (TypeError, ValueError):
                    alpha = None
```

```
            if (
                alpha is not None
                and math.isfinite(alpha)
                and alpha >= 0
            ):
                message["alpha_power"] = alpha

        # A classifier command is optional.
        predicted_class = data.get("predicted_class")

        if predicted_class in ("left_hand", "right_hand"):
            message["predicted_class"] = predicted_class

        if (
            message["alpha_power"] is None
            and message["predicted_class"] is None
        ):
            continue

        latest_data = message
        await broadcast(message)

  except WebSocketDisconnect:
      print("Python disconnected")

@app.websocket("/ws/unity")
async def unity_stream(websocket: WebSocket):
  await websocket.accept()

  async with clients_lock:
      unity_clients.add(websocket)

  print("Unity connected")

  try:
      await websocket.send_json(latest_data.copy())

      while True:
          await websocket.receive_text()

  except WebSocketDisconnect:
      pass
  finally:
      async with clients_lock:
          unity_clients.discard(websocket)

      print("Unity disconnected")
train_eeg.py:

from pathlib import Path
import warnings

import joblib
import mne
import numpy as np
import pandas as pd

from mne.decoding import CSP
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import LeaveOneGroupOut, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.metrics import balanced_accuracy_score, accuracy_score

# ==========================
# CONFIGURATION
# ==========================

DATASET_DIR = Path(r"C:\nm000158_v1.0.5")
MODEL_PATH = Path("motor_imagery_model.joblib")

CHANNELS = ["C3", "Cz", "C4"]
CLASS_NAMES = ["left_hand", "right_hand"]

WINDOW_START = 0.5
WINDOW_END = 3.5

SKIP_SUBJECTS = {"sub-5", "sub-15", "sub-49"}
```

```
mne.set_log_level("ERROR")
warnings.filterwarnings("ignore", category=RuntimeWarning)

def find_channel(raw, target):
    """Find an EEG channel by its name."""
    target = target.upper()

    for name in raw.ch_names:
        normalized = (
            name.upper()
            .replace(" ", "")
            .replace("-", "")
            .replace("_", "")
        )

        if normalized == target or normalized.endswith(target):
            return name

    return None

def load_dataset():
    X = []
    y = []
    groups = []

    files = sorted(
        DATASET_DIR.glob("sub-*/ses-*/eeg/*_eeg.bdf")
    )

    if not files:
        raise FileNotFoundError(
            f"No BDF files found in {DATASET_DIR}\n"
            "Check DATASET_DIR and the dataset folder structure."
        )

    print(f"Found BDF files: {len(files)}")

    for bdf_path in files:
        subject = bdf_path.parts[-4]

        if subject in SKIP_SUBJECTS:
            print(f"Skipping {subject}")
            continue

        events_path = bdf_path.with_name(
            bdf_path.name.replace("_eeg.bdf", "_events.tsv")
        )

        if not events_path.exists():
            print(f"Events file not found: {events_path}")
            continue

        try:
            events = pd.read_csv(events_path, sep="\t")

            required = {"onset", "trial_type"}
            if not required.issubset(events.columns):
                print(f"Unexpected event columns: {events_path}")
                continue

            raw = mne.io.read_raw_bdf(
                str(bdf_path),
                preload=True,
                verbose=False,
            )

            channel_map = {
                channel: find_channel(raw, channel)
                for channel in CHANNELS
            }

            if any(value is None for value in channel_map.values()):
                print(
                    f"Required channels not found for {subject}: "
                    f"{channel_map}"
                )
                continue

            raw.pick_channels(list(channel_map.values()))
```

```
            # Rename channels consistently across recordings.
            reverse_map = {
                actual: standard
                for standard, actual in channel_map.items()
            }
            raw.rename_channels(reverse_map)

            # Motor imagery frequency band.
            raw.filter(
                l_freq=8.0,
                h_freq=30.0,
                verbose=False,
            )

            sfreq = raw.info["sfreq"]
            subject_trials = 0

            for _, event in events.iterrows():
                label = str(event["trial_type"]).strip().lower()

                if label not in CLASS_NAMES:
                    continue

                onset = float(event["onset"])
                start_time = onset + WINDOW_START
                end_time = onset + WINDOW_END

                start_sample = int(round(start_time * sfreq))
                end_sample = int(round(end_time * sfreq))

                if (
                    start_sample < 0
                    or end_sample > raw.n_times
                    or end_sample <= start_sample
                ):
                    continue

                epoch = raw.get_data(
                    start=start_sample,
                    stop=end_sample,
                )

                expected_samples = int(
                    round((WINDOW_END - WINDOW_START) * sfreq)
                )

                if epoch.shape[1] != expected_samples:
                    continue

                X.append(epoch)
                y.append(label)
                groups.append(subject)
                subject_trials += 1

            print(f"{subject}: loaded {subject_trials} trials")

        except Exception as exc:
            print(f"Error processing {bdf_path}: {exc}")

    if not X:
        raise RuntimeError(
            "No trials loaded. Check event labels, channel names "
            "and dataset structure."
        )

    X = np.asarray(X, dtype=np.float64)
    y = np.asarray(y)
    groups = np.asarray(groups)

    print("\nDataset loaded")
    print(f"Trials: {len(X)}")
    print(f"EEG shape: {X.shape}")
    print(f"Subjects: {len(np.unique(groups))}")
    print(f"Classes: {dict(zip(*np.unique(y, return_counts=True)))}")

    return X, y, groups

def create_model():
```

```
    return Pipeline([
        (
            "csp",
            CSP(
                n_components=2,
                reg=None,
                log=True,
                norm_trace=False,
            ),
        ),
        (
            "lda",
            LinearDiscriminantAnalysis(),
        ),
    ])

def main():
    X, y, groups = load_dataset()

    if len(np.unique(y)) != 2:
        raise RuntimeError(
            "Expected both left_hand and right_hand classes."
        )

    unique_subjects = np.unique(groups)

    if len(unique_subjects) < 2:
        raise RuntimeError(
            "At least two subjects are required for "
            "leave-one-subject-out evaluation."
        )

    print("\nEvaluating model by held-out subject...")

    model = create_model()
    cv = LeaveOneGroupOut()

    scores = cross_val_score(
        model,
        X,
        y,
        groups=groups,
        cv=cv,
        scoring="accuracy",
        n_jobs=1,
    )

    print(f"Mean accuracy: {scores.mean():.3f}")
    print(f"Std accuracy:  {scores.std():.3f}")
    print(f"Per-fold scores: {np.round(scores, 3)}")

    # Train the final pipeline on all available trials.
    print("\nTraining final model on all trials...")
    model.fit(X, y)

    artifact = {
        "model": model,
        "channels": CHANNELS,
        "classes": model.named_steps["lda"].classes_.tolist(),
        "window_start": WINDOW_START,
        "window_end": WINDOW_END,
        "filter_band": [8.0, 30.0],
        "sampling_frequency": 500,
        "training_subjects": unique_subjects.tolist(),
        "evaluation": {
            "method": "LeaveOneGroupOut",
            "mean_accuracy": float(scores.mean()),
            "std_accuracy": float(scores.std()),
        },
    }

    joblib.dump(artifact, MODEL_PATH)

    print(f"\nModel saved to: {MODEL_PATH.resolve()}")
    print("Done.")

if __name__ == "__main__":
    main()
```

replay_eeg.py:

```
import asyncio
from pathlib import Path

import joblib
import mne
import numpy as np
import pandas as pd
import websockets

# ==========================
# CONFIGURATION
# ==========================

MODEL_PATH = Path("motor_imagery_model.joblib")

DATASET_DIR = Path(r"C:\nm000158_v1.0.5")

BDF_PATH = (
    DATASET_DIR
    / "sub-1"
    / "ses-0"
    / "eeg"
    / "sub-1_ses-0_task-imagery_run-0_eeg.bdf"
)

EVENTS_PATH = BDF_PATH.with_name(
    BDF_PATH.name.replace("_eeg.bdf", "_events.tsv")
)

SERVER_URL = "ws://127.0.0.1:8000/ws/python"

REPLAY_SPEED = 1.0
PAUSE_BETWEEN_PREDICTIONS = 1.0

mne.set_log_level("ERROR")

def find_channel(raw, target):
    target = target.upper()

    for name in raw.ch_names:
        normalized = (
            name.upper()
            .replace(" ", "")
            .replace("-", "")
            .replace("_", "")
        )

        if normalized == target or normalized.endswith(target):
            return name

    return None

def load_recording(artifact):
    if not BDF_PATH.exists():
        raise FileNotFoundError(
            f"EEG file not found:\n{BDF_PATH}"
        )

    if not EVENTS_PATH.exists():
        raise FileNotFoundError(
            f"Events file not found:\n{EVENTS_PATH}"
        )

    print(f"Loading EEG recording:\n{BDF_PATH}")

    raw = mne.io.read_raw_bdf(
        str(BDF_PATH),
        preload=True,
        verbose=False,
    )

    channel_names = artifact["channels"]
    channel_map = {
        channel: find_channel(raw, channel)
        for channel in channel_names
    }
```

```
    if any(value is None for value in channel_map.values()):
        raise RuntimeError(
            f"Required EEG channels are missing: {channel_map}"
        )

    raw.pick_channels(list(channel_map.values()))

    raw.rename_channels({
        actual: standard
        for standard, actual in channel_map.items()
    })

    raw.filter(
        l_freq=artifact["filter_band"][0],
        h_freq=artifact["filter_band"][1],
        verbose=False,
    )

    events = pd.read_csv(EVENTS_PATH, sep="\t")

    required = {"onset", "trial_type"}
    if not required.issubset(events.columns):
        raise RuntimeError(
            f"Unexpected event columns: {events.columns.tolist()}"
        )

    return raw, events

async def main():
    if not MODEL_PATH.exists():
        raise FileNotFoundError(
            f"Model not found: {MODEL_PATH.resolve()}\n"
            "Run python train_eeg.py first."
        )

    artifact = joblib.load(MODEL_PATH)
    model = artifact["model"]

    raw, events = load_recording(artifact)

    sfreq = raw.info["sfreq"]
    window_start = float(artifact["window_start"])
    window_end = float(artifact["window_end"])

    print("\nModel loaded.")
    print(f"Classes: {artifact['classes']}")
    print(f"Recording sampling rate: {sfreq} Hz")
    print(f"Replay speed: {REPLAY_SPEED}x")
    print(f"Connecting to {SERVER_URL} ...")

    async with websockets.connect(SERVER_URL) as websocket:
        print("Connected to FastAPI.")
        print("Starting EEG playback...\n")

        sent = 0

        for _, event in events.iterrows():
            label = str(event["trial_type"]).strip().lower()

            if label not in ("left_hand", "right_hand"):
                continue

            onset = float(event["onset"])
            start_time = onset + window_start
            end_time = onset + window_end

            start_sample = int(round(start_time * sfreq))
            end_sample = int(round(end_time * sfreq))

            if (
                start_sample < 0
                or end_sample > raw.n_times
                or end_sample <= start_sample
            ):
                continue

            epoch = raw.get_data(
                start=start_sample,
                stop=end_sample,
```

```
            )

            expected_samples = int(
                round((window_end - window_start) * sfreq)
            )

            if epoch.shape[1] != expected_samples:
                continue

            # CSP expects shape: trials x channels x samples.
            prediction = model.predict(epoch[np.newaxis, :, :])[0]

            # Send the model's prediction, not the event's true label.
            message = {
                "predicted_class": str(prediction),
                "source": "dataset_replay",
            }

            await websocket.send(
                __import__("json").dumps(message)
            )

            sent += 1

            print(
                f"Trial {sent:02d} | "
                f"onset={onset:.2f}s | "
                f"prediction={prediction} | "
                f"dataset label={label}"
            )

            await asyncio.sleep(
                PAUSE_BETWEEN_PREDICTIONS / REPLAY_SPEED
            )

        print(f"\nPlayback finished. Predictions sent: {sent}")

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\nPlayback stopped.")
```